



```
In [1]: # --- Experiment: Implementing Transposed Convolution, Object Detection, and S
# Let's work on several tasks that will help you understand the details of con

# --- Task 1: Implement Transposed Convolution using numpy ---
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

def transposed_convolution(input_matrix, kernel, stride=1, padding=0):
    """
    Implement a basic transposed convolution (also called deconvolution) using

    Parameters:
    - input_matrix: 2D input feature map (numpy array), shape (H_in, W_in).
    - kernel: 2D convolution kernel (numpy array), shape (K_h, K_w).
    - stride: integer "stride" of the deconvolution.
    - padding: integer "padding" of the deconvolution.

    Returns:
    - 2D numpy array: Output of the transposed convolution.
    """
    input_matrix = np.asarray(input_matrix)
    kernel = np.asarray(kernel)

    if input_matrix.ndim != 2 or kernel.ndim != 2:
        raise ValueError("input_matrix and kernel must both be 2D arrays")
    if not isinstance(stride, int) or stride <= 0:
        raise ValueError("stride must be a positive integer")
    if padding < 0:
        raise ValueError("padding must be a non-negative integer")

    H_in, W_in = input_matrix.shape
    K_h, K_w = kernel.shape

    # 1. Insert zeros between and around input pixels
    H_up = (H_in - 1)*stride - 2*padding + 2*K_h - 1
    W_up = (W_in - 1)*stride - 2*padding + 2*K_w - 1
    padded = np.zeros((H_up, W_up), dtype=input_matrix.dtype)

    # final padding at h and w around pixels, to determine start and end point
    fph, fpw = K_h - padding - 1, K_w - padding - 1
    padded[fph:H_up-fph:stride, fpw:W_up-fpw:stride] = input_matrix
    print(f"{'#' * 30}\nPadded matrix and its shape:\n", padded, padded.shape)

    # 2. Full 2D convolution: output size = H_up + K_h - 1, W_up + K_w - 1
    H_out_final = H_up - K_h + 1 # i.e. (H_in-1)*s-2p+k
    W_out_final = W_up - K_w + 1
    output_final = np.zeros((H_out_final, W_out_final), dtype=input_matrix.dty

    # 3. normal convolution
    for i in range(H_out_final):
        for j in range(W_out_final):
            region = padded[i:i + K_h, j:j + K_w]
```

```

        output_final[i, j] += np.sum(region * kernel)
    return output_final

# Test the function with a simple input matrix and kernel
input_matrix = np.array([[1, 2], [3, 4]], dtype=np.float64)
kernel = np.array([[0, 1, 0], [1, 1, 1], [0, 1, 0]], dtype=np.float64)
output = transposed_convolution(input_matrix, kernel, stride=2, padding=1)
print(f"{'#'*30}\n Test Sample 1: \n", output)

# more test samples
input_matrix = np.array([[2, 5, 6], [1, 3, 4]], dtype=np.float64)
kernel = np.array([[0, 1, 0], [1, 1, 1], [0, 1, 0]], dtype=np.float64)
output = transposed_convolution(input_matrix, kernel, stride=2, padding=1)
print(f"{'#'*30}\n Test Sample 2: \n", output)
output = transposed_convolution(input_matrix, kernel, stride=3, padding=1)
print(f"{'#'*30}\n Test Sample 3: \n", output)
output = transposed_convolution(input_matrix, kernel, stride=1, padding=0)
print(f"{'#'*30}\n Test Sample 4: \n", output)
output = transposed_convolution(input_matrix, kernel, stride=2, padding=0)
print(f"{'#'*30}\n Test Sample 5: \n", output)

# Plot input and output
plt.subplot(1, 2, 1)
plt.title("Input Matrix")
plt.imshow(input_matrix, cmap='gray')

plt.subplot(1, 2, 2)
plt.title("Transposed Convolution Output")
plt.imshow(output, cmap='gray')
plt.show()

```

```
#####  
Padded matrix and its shape:  
[[0. 0. 0. 0. 0.]  
 [0. 1. 0. 2. 0.]  
 [0. 0. 0. 0. 0.]  
 [0. 3. 0. 4. 0.]  
 [0. 0. 0. 0. 0.]] (5, 5)
```

```
#####  
Test Sample 1:  
[[1. 3. 2.]  
 [4. 0. 6.]  
 [3. 7. 4.]]
```

```
#####  
Padded matrix and its shape:  
[[0. 0. 0. 0. 0. 0. 0.]  
 [0. 2. 0. 5. 0. 6. 0.]  
 [0. 0. 0. 0. 0. 0. 0.]  
 [0. 1. 0. 3. 0. 4. 0.]  
 [0. 0. 0. 0. 0. 0. 0.]] (5, 7)
```

```
#####  
Test Sample 2:  
[[ 2.  7.  5. 11.  6.]  
 [ 3.  0.  8.  0. 10.]  
 [ 1.  4.  3.  7.  4.]]
```

```
#####  
Padded matrix and its shape:  
[[0. 0. 0. 0. 0. 0. 0. 0. 0.]  
 [0. 2. 0. 0. 5. 0. 0. 6. 0.]  
 [0. 0. 0. 0. 0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0. 0. 0. 0. 0.]  
 [0. 1. 0. 0. 3. 0. 0. 4. 0.]  
 [0. 0. 0. 0. 0. 0. 0. 0. 0.]] (6, 9)
```

```
#####  
Test Sample 3:  
[[2. 2. 5. 5. 5. 6. 6.]  
 [2. 0. 0. 5. 0. 0. 6.]  
 [1. 0. 0. 3. 0. 0. 4.]  
 [1. 1. 3. 3. 3. 4. 4.]]
```

```
#####  
Padded matrix and its shape:  
[[0. 0. 0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0. 0. 0.]  
 [0. 0. 2. 5. 6. 0. 0.]  
 [0. 0. 1. 3. 4. 0. 0.]  
 [0. 0. 0. 0. 0. 0. 0.]  
 [0. 0. 0. 0. 0. 0. 0.]] (6, 7)
```

```
#####  
Test Sample 4:  
[[ 0.  2.  5.  6.  0.]  
 [ 2.  8. 16. 15.  6.]  
 [ 1.  6. 13. 13.  4.]  
 [ 0.  1.  3.  4.  0.]]
```

```
#####  
Padded matrix and its shape:
```

```

[[0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 2. 0. 5. 0. 6. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 1. 0. 3. 0. 4. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0. 0. 0. 0. 0.]] (7, 9)

```

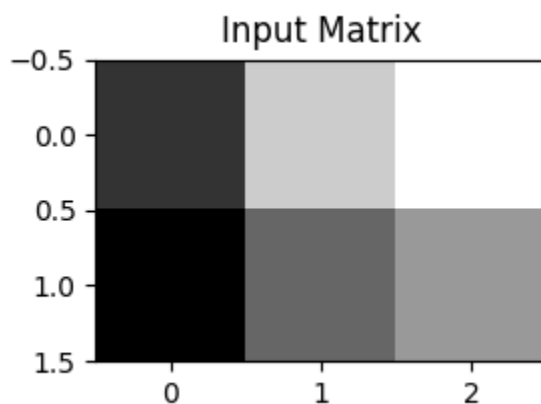
#####

Test Sample 5:

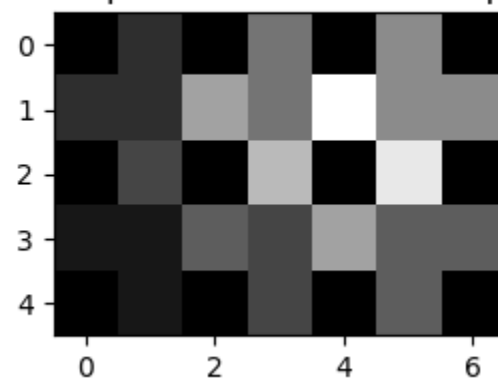
```

[[ 0.  2.  0.  5.  0.  6.  0.]
 [ 2.  2.  7.  5. 11.  6.  6.]
 [ 0.  3.  0.  8.  0. 10.  0.]
 [ 1.  1.  4.  3.  7.  4.  4.]
 [ 0.  1.  0.  3.  0.  4.  0.]]

```



Transposed Convolution Output



```

In [2]: # @title Object detection utils, libraries and predefined labels of COCO datas
import torch

# COCO dataset classes, N/As are unused ids
COCO_CLASSES = (
    'N/A', 'person', 'bicycle', 'car', 'motorcycle', 'airplane', 'bus',
    'train', 'truck', 'boat', 'traffic light', 'fire hydrant', 'N/A',
    'stop sign', 'parking meter', 'bench', 'bird', 'cat', 'dog', 'horse',
    'sheep', 'cow', 'elephant', 'bear', 'zebra', 'giraffe', 'N/A', 'backpack',
    'umbrella', 'N/A', 'N/A', 'handbag', 'tie', 'suitcase', 'frisbee', 'skis',
    'snowboard', 'sports ball', 'kite', 'baseball bat', 'baseball glove',
    'skateboard', 'surfboard', 'tennis racket', 'bottle', 'N/A', 'wine glass',
    'cup', 'fork', 'knife', 'spoon', 'bowl', 'banana', 'apple', 'sandwich',
    'orange', 'broccoli', 'carrot', 'hot dog', 'pizza', 'donut', 'cake',
    'chair', 'couch', 'potted plant', 'bed', 'N/A', 'dining table', 'N/A',
    'N/A', 'toilet', 'N/A', 'tv', 'laptop', 'mouse', 'remote', 'keyboard',
    'cell phone', 'microwave', 'oven', 'toaster', 'sink', 'refrigerator', 'N/A',
    'book', 'clock', 'vase', 'scissors', 'teddy bear', 'hair drier',
    'toothbrush'
)

# for output bounding box post-processing
def box_cxxywh_to_xyxy(x):
    x_c, y_c, w, h = x.unbind(1)
    b = [(x_c - 0.5 * w), (y_c - 0.5 * h),

```

```

        (x_c + 0.5 * w), (y_c + 0.5 * h)]
    return torch.stack(b, dim=1)

def rescale_bboxes(out_bbox, size):
    img_w, img_h = size
    b = box_cxcywh_to_xyxy(out_bbox)
    b = b * torch.tensor([img_w, img_h, img_w, img_h], dtype=torch.float32)
    return b

```

```

In [3]: import torch
import torchvision.transforms as T
import matplotlib.pyplot as plt
from PIL import Image
import requests

# Load a pretrained DETR model and use it for object detection
def detect_objects_with_detr(image_path):
    image = Image.open(image_path).convert("RGB")
    model = torch.hub.load('facebookresearch/detr', 'detr_resnet50', pretrained=True)
    model.eval();

    # Step 1: define the preprocessing
    transform = T.Compose([
        T.Resize(800),
        T.ToTensor(),
        T.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
    ])

    # Bonus 1: use lists and hooks to store the intermediate outputs
    conv_features, dec_attn_weights = [], []
    hooks = [
        model.backbone[-2].register_forward_hook(
            lambda self, input, output: conv_features.append(output)
        ),
        model.transformer.decoder.layers[-1].multihead_attn.register_forward_hook(
            lambda self, input, output: dec_attn_weights.append(output[1])
        ),
    ]

    # Step 2: apply preprocessing and model inference
    image_tensor = transform(image).unsqueeze(0)
    outputs = model(image_tensor)

    # Step 3: post-processing, only keep high confidence predictions, rescale
    probas = outputs['pred_logits'].softmax(-1)[0, :, :-1]
    keep = probas.max(-1).values > 0.9
    prob = probas[keep].detach().numpy()
    # convert boxes from [0; 1] to image scales
    bboxes_scaled = rescale_bboxes(outputs['pred_boxes'][0, keep], image.size)

    # Step 4: Plot bounding boxes
    plt.figure(figsize=(16,10))
    plt.imshow(image)

```

```

ax = plt.gca()
# colors for visualization
COLORS = ['red', 'yellow', 'blue']
colors = COLORS * 100
for p, (xmin, ymin, xmax, ymax), c in zip(prob, bboxes_scaled, colors):
    ax.add_patch(plt.Rectangle((xmin, ymin), xmax - xmin, ymax - ymin,
                               fill=False, color=c, linewidth=3))

    cl = p.argmax()
    text = f'{COCO_CLASSES[cl]}: {p[cl]:0.2f}'
    ax.text(xmin, ymin, text, fontsize=15,
            bbox=dict(facecolor='yellow', alpha=0.5))
plt.axis('off')
plt.show()

# Bonus 2: fetch and visualize intermediate feature and attention map
for hook in hooks:
    hook.remove()
conv_features = conv_features[0]
dec_attn_weights = dec_attn_weights[0]
# get the feature map shape
h, w = conv_features['0'].tensors.shape[-2:]
fig, axs = plt.subplots(ncols=len(bboxes_scaled), nrows=2, figsize=(16, 8))
for idx, ax_i, (xmin, ymin, xmax, ymax) in zip(keep.nonzero(), axs.T, bboxes_scaled):
    ax = ax_i[0]
    ax.imshow(dec_attn_weights[0, idx].view(h, w).detach().numpy())
    ax.axis('off')
    ax.set_title(f'query id: {idx.item()}', fontsize=15)
    ax = ax_i[1]
    ax.imshow(image)
    ax.add_patch(plt.Rectangle((xmin, ymin), xmax - xmin, ymax - ymin,
                               fill=False, color='blue', linewidth=3))

    ax.axis('off')
    ax.set_title(COCO_CLASSES[probas[idx].argmax()], fontsize=15)
fig.tight_layout()

# Call detect_objects_with_detr with an example image
detect_objects_with_detr("img110.png")

```

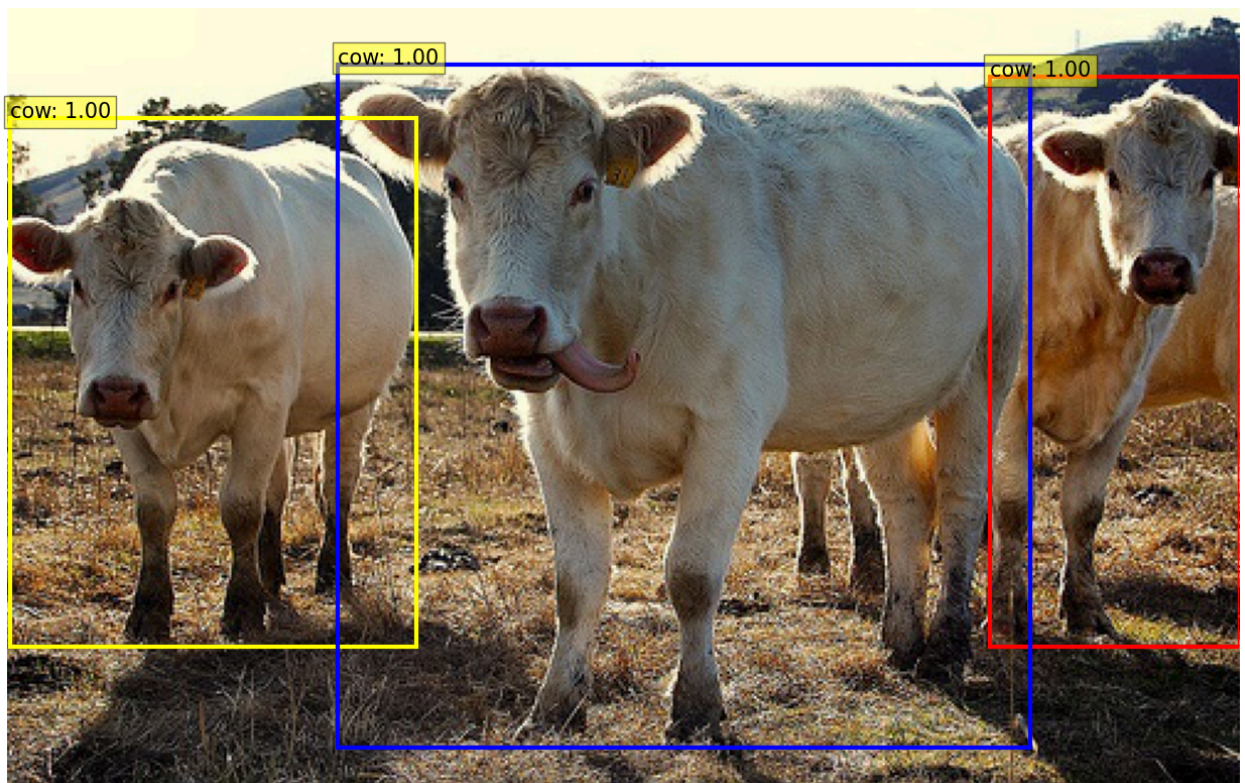
Using cache found in /root/.cache/torch/hub/facebookresearch_detr_main

/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated since 0.13 and may be removed in the future, please use 'weights' instead.

```
warnings.warn(
```

/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=ResNet50\_Weights.IMAGENET1K\_V1`. You can also use `weights=ResNet50\_Weights.DEFAULT` to get the most up-to-date weights.

```
warnings.warn(msg)
```

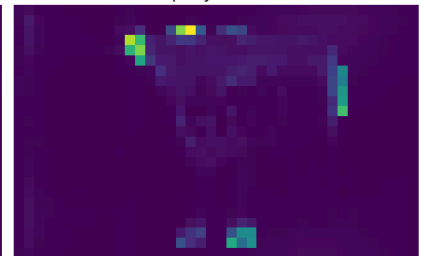
query id: 12



query id: 61



query id: 71



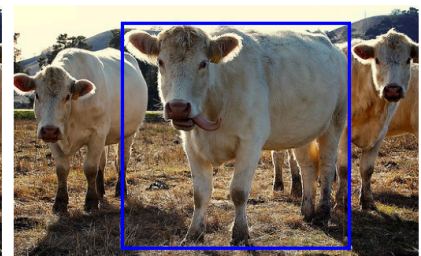
cow



cow



cow



```
In [4]: # --- Task 3: Use Pretrained U-Net for Semantic Segmentation ---
import torchvision.models.segmentation as seg_models

def segment_image_with_unet(image_path):
    model = seg_models.deeplabv3_resnet101(pretrained=True)
    model.eval()

    transform = T.Compose([
        T.ToTensor(),
        T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    ])
```

```

image = Image.open(image_path).convert("RGB")
image = transform(image).unsqueeze(0)

with torch.no_grad():
    output = model(image)

output_predictions = output['out'][0].argmax(0)

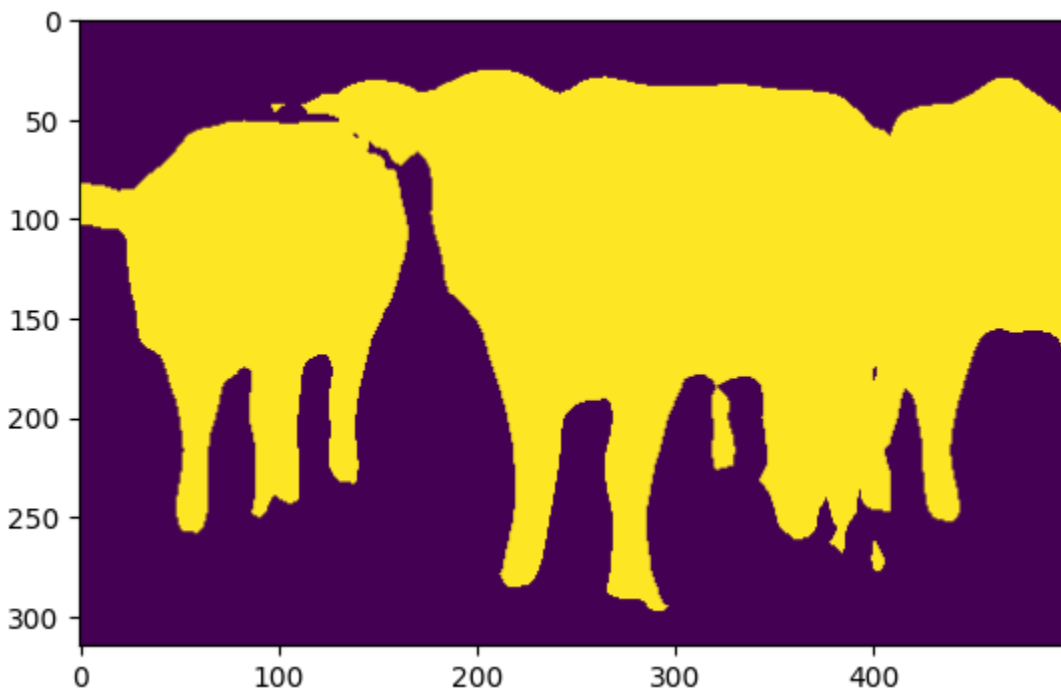
plt.imshow(output_predictions.cpu().numpy())
plt.show()

# Call segment_image_with_unet with an example image
segment_image_with_unet("img110.png")

```

/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `None` for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=DeepLabV3\_ResNet101\_Weights.COCO\_WITH\_VOC\_LABELS\_V1`. You can also use `weights=DeepLabV3\_ResNet101\_Weights.DEFAULT` to get the most up-to-date weights.

warnings.warn(msg)



In [5]: *# Here are some transposed convolution examples using PyTorch, if you're inter*

```

import torch

inputs = np.array([[1, 2], [3, 4]], dtype=np.float64)
kernels = np.array([[0, 1, 0], [1, 1, 1], [0, 1, 0]], dtype=np.float64)
inputs_torch = torch.from_numpy(inputs)[None, None, ...]
kernels_torch = torch.from_numpy(kernels)[None, None, ...]
conv_transposed2d_obj = torch.nn.ConvTranspose2d(
    in_channels=kernels.shape[0],

```



```

        out_channels=kernels.shape[1],
        kernel_size=kernels.shape[:2],
        stride=2,
        padding=1,
        output_padding=0,
        bias=False
    )
    conv_transposed2d_obj.weight = torch.nn.Parameter(kernels_torch)
    outputs_torch = conv_transposed2d_obj(inputs_torch)
    print(f"{'#'*30}\n Test Sample 1: \n", outputs_torch.detach().numpy())

inputs = np.array([[2, 5, 6], [1, 3, 4]], dtype=np.float64)
kernels = np.array([[0, 1, 0], [1, 1, 1], [0, 1, 0]], dtype=np.float64)
inputs_torch = torch.from_numpy(inputs)[None, None, ...]
kernels_torch = torch.from_numpy(kernels)[None, None, ...]

conv_transposed2d_obj = torch.nn.ConvTranspose2d(
    in_channels=kernels.shape[0], out_channels=kernels.shape[1],
    kernel_size=kernels.shape[:2], stride=2, padding=1, bias=False
)
conv_transposed2d_obj.weight = torch.nn.Parameter(kernels_torch)
outputs_torch = conv_transposed2d_obj(inputs_torch)
print(f"{'#'*30}\n Test Sample 2: \n", outputs_torch.detach().numpy())

conv_transposed2d_obj = torch.nn.ConvTranspose2d(
    in_channels=kernels.shape[0], out_channels=kernels.shape[1],
    kernel_size=kernels.shape[:2], stride=3, padding=1, bias=False
)
conv_transposed2d_obj.weight = torch.nn.Parameter(kernels_torch)
outputs_torch = conv_transposed2d_obj(inputs_torch)
print(f"{'#'*30}\n Test Sample 3: \n", outputs_torch.detach().numpy())

conv_transposed2d_obj = torch.nn.ConvTranspose2d(
    in_channels=kernels.shape[0], out_channels=kernels.shape[1],
    kernel_size=kernels.shape[:2], stride=1, padding=0, bias=False
)
conv_transposed2d_obj.weight = torch.nn.Parameter(kernels_torch)
outputs_torch = conv_transposed2d_obj(inputs_torch)
print(f"{'#'*30}\n Test Sample 4: \n", outputs_torch.detach().numpy())

conv_transposed2d_obj = torch.nn.ConvTranspose2d(
    in_channels=kernels.shape[0], out_channels=kernels.shape[1],
    kernel_size=kernels.shape[:2], stride=2, padding=0, bias=False
)
conv_transposed2d_obj.weight = torch.nn.Parameter(kernels_torch)
outputs_torch = conv_transposed2d_obj(inputs_torch)
print(f"{'#'*30}\n Test Sample 5: \n", outputs_torch.detach().numpy())

```

#####

Test Sample 1:

[[[1. 3. 2.]
[4. 0. 6.]
[3. 7. 4.]]]

#####

Test Sample 2:

[[[2. 7. 5. 11. 6.]
[3. 0. 8. 0. 10.]
[1. 4. 3. 7. 4.]]]

#####

Test Sample 3:

[[[2. 2. 5. 5. 5. 6. 6.]
[2. 0. 0. 5. 0. 0. 6.]
[1. 0. 0. 3. 0. 0. 4.]
[1. 1. 3. 3. 3. 4. 4.]]]

#####

Test Sample 4:

[[[0. 2. 5. 6. 0.]
[2. 8. 16. 15. 6.]
[1. 6. 13. 13. 4.]
[0. 1. 3. 4. 0.]]]

#####

Test Sample 5:

[[[0. 2. 0. 5. 0. 6. 0.]
[2. 2. 7. 5. 11. 6. 6.]
[0. 3. 0. 8. 0. 10. 0.]
[1. 1. 4. 3. 7. 4. 4.]
[0. 1. 0. 3. 0. 4. 0.]]]